Introduction to AI

Uninformed Search (Reminder)

Alex Shleyfman and Gal A. Kaminka

Dept. of Computer Science, Bar-Ilan University

Winter 2026/2027

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

From problems to models

Deterministic planning

Formalism assumptions

Back to our problems

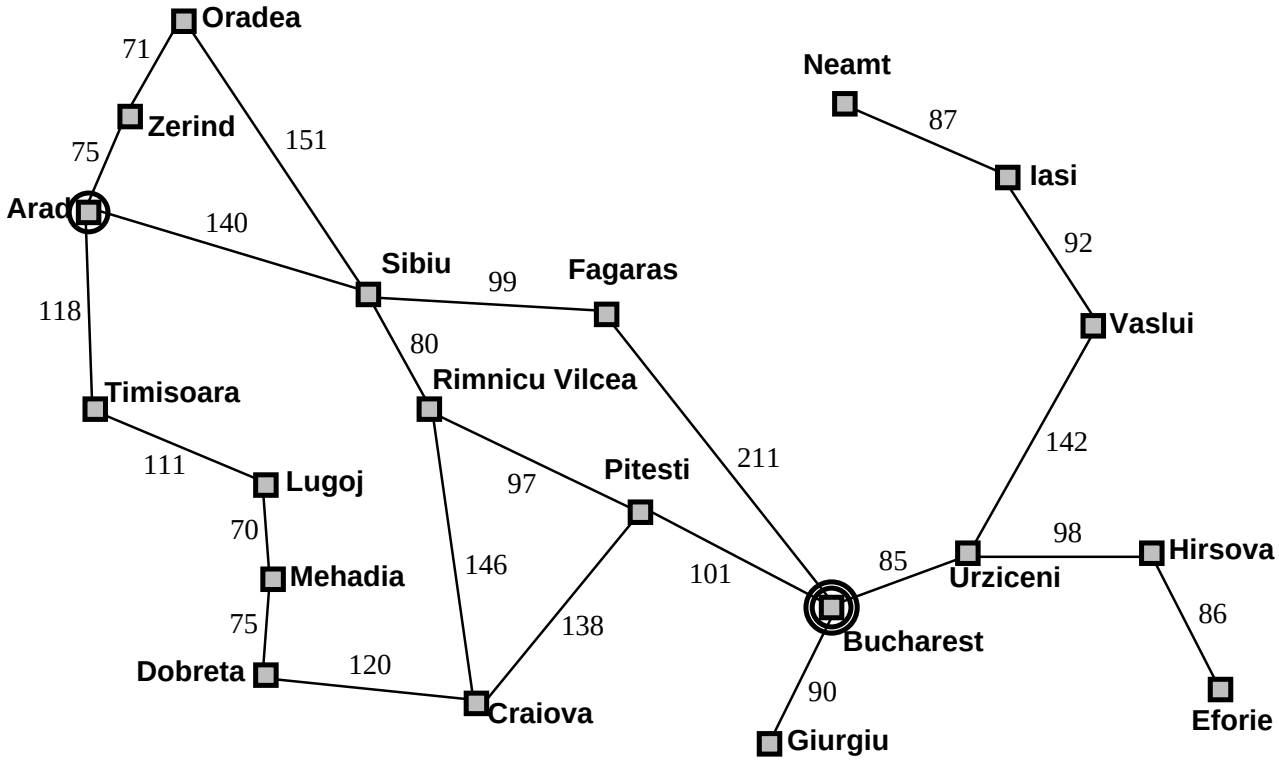
State-space search

Uninformed search

Example: A trip in Romania

Task

On holiday in Romania; currently in Arad.
Flight leaves tomorrow from Bucharest



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

From problems to models

Deterministic planning

Formalism assumptions

Back to our problems

State-space search

Uninformed search

Example: A trip in Romania

Task

On holiday in Romania; currently in Arad.
Flight leaves tomorrow from Bucharest

- Formulate goal: be in Bucharest
- Formulate problem:
  - states: various cities
  - actions: drive between cities
- Find solution: sequence of cities,
  e.g., Arad, Sibiu, Fagaras, Bucharest

You probably know how to solve this one, but ...

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Example: A trip in Romania

Task

On holiday in Romania; currently in Arad.
Flight leaves tomorrow from Bucharest

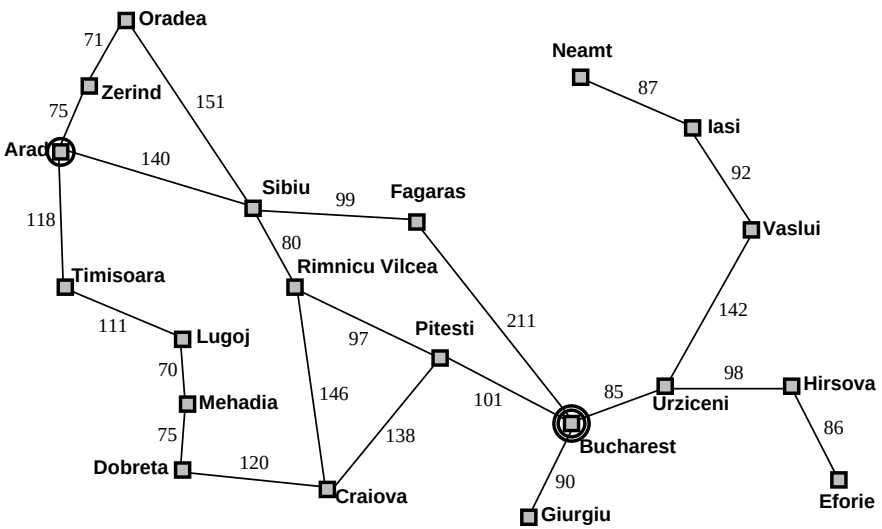
- Formulate goal: be in Bucharest
- Formulate problem:
  - states: various cities
  - actions: drive between cities
- Find solution: sequence of cities,
  e.g., Arad, Sibiu, Fagaras, Bucharest

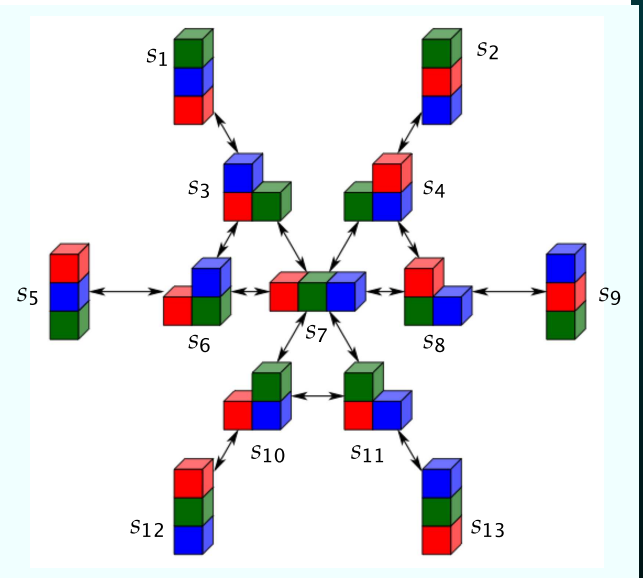
You probably know how to solve this one, but ...

Search Problems

What is in common?

- All these problems deal with action selection or control
- Some notion of problem state
- (Often) specification of initial state and/or goal state
- Legal moves or actions that allow transition from states to other states





Introduction to AI

Alex Shleyfman and Gal A. Kaminka

From problems to models

Deterministic planning

Formalism assumptions

Back to our problems

State-space search

Uninformed search

Sequence-Search Problems

For now focus on:

- Plans (aka solutions) are sequences of moves that transform the initial state into the goal state
- Intuitively, not all solutions are equally desirable

What is our task?

1. Find out whether there is a solution
2. Find any solution
3. Find an optimal solution
4. Find near-optimal solution
5. Fixed amount of time, find best solution possible
6. Find solution that satisfy property $\aleph$ (what is $\aleph$? you choose!)

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Sequence-Search Problems

What is our task?

1. Find out whether there is a solution
2. Find any solution
3. Find an optimal solution
4. Find near-optimal solution
5. Fixed amount of time, find best solution possible
6. Find solution that satisfy property $\aleph$ (what is $\aleph$? you choose!)

- ♠ While all these tasks sound related, they are very different. The best suited techniques are almost disjoint.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Three Approaches to Problem Solving

1. programming: specify control by hand
2. model-oriented solving:
   specify problem by hand, derive control automatically
3. learning: learn control from experience

Planning is a form of model-oriented problem solving

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

State Model for Deterministic planning

- finite state space S
- an initial state $s_0 \in S$
- a set $S_G \subseteq S$ of goal states
- applicable actions $A(s) \subseteq A$ for $s \in S$
- a transition function $s' = f(a, s)$ for $a \in A(s)$
- a cost function $c : A \to [0, \infty)$

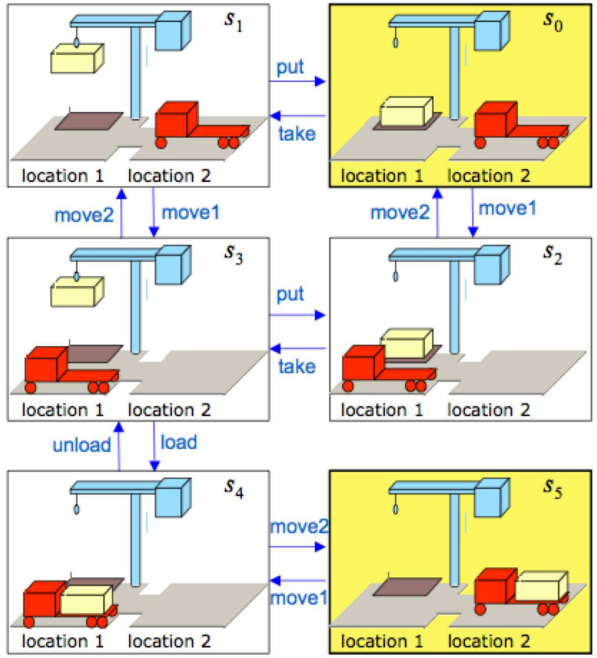
A **solution** is a sequence of applicable actions that maps s_0 into S_G
An **optimal solution** minimizes c

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

State Model for Deterministic planning

- finite state space S
- an initial state $s_0 \in S$
- a set $S_G \subseteq S$ of goal states
- applicable actions
  $A(s) \subseteq A$ for $s \in S$
- a transition function
  $s' = f(a, s)$ for $a \in A(s)$
- a cost function $c : A \to [0, \infty)$

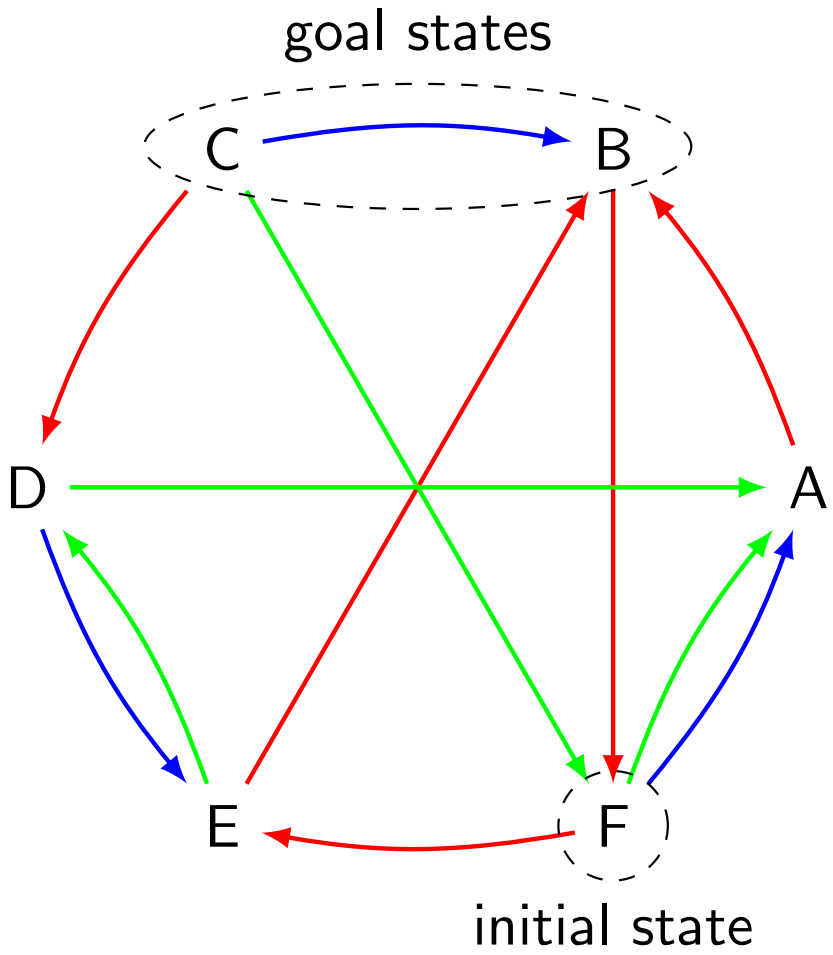
A **solution** is a sequence of applicable actions that maps s_0 into S_G
An **optimal solution** minimizes c



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Transition graph

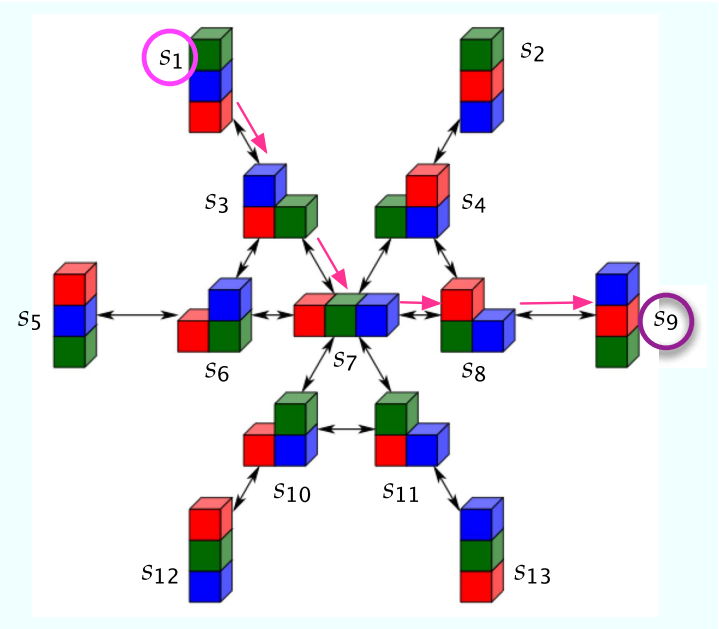
Small example



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Why planning is difficult?

- Solutions to planning problems are paths from an initial state to a goal state in the transition graph
- Dijkstra's algorithm solves this problem in $O(|V| \log(|V|) + |E|)$
- **Can we just go home?**



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

From problems to models

Deterministic planning

Formalism assumptions

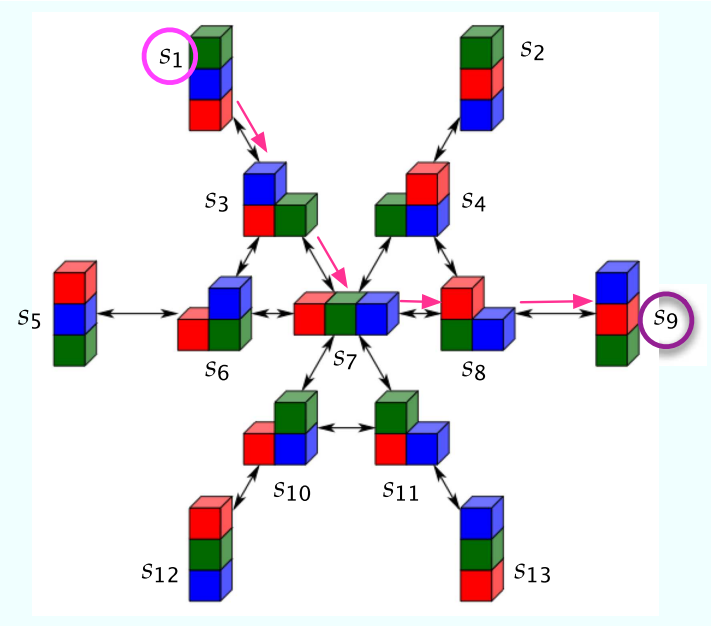
Back to our problems

State-space search

Uninformed search

Why planning is difficult?

- Solutions to planning problems are paths from an initial state to a goal state in the transition graph
- Dijkstra's algorithm solves this problem in $O(|V| \log(|V|) + |E|)$
- **Can we just go home?**
- Not exactly $\Rightarrow$ $|V|$ of our interest is 10^{10}, 10^{20}, 10^{100}, ...

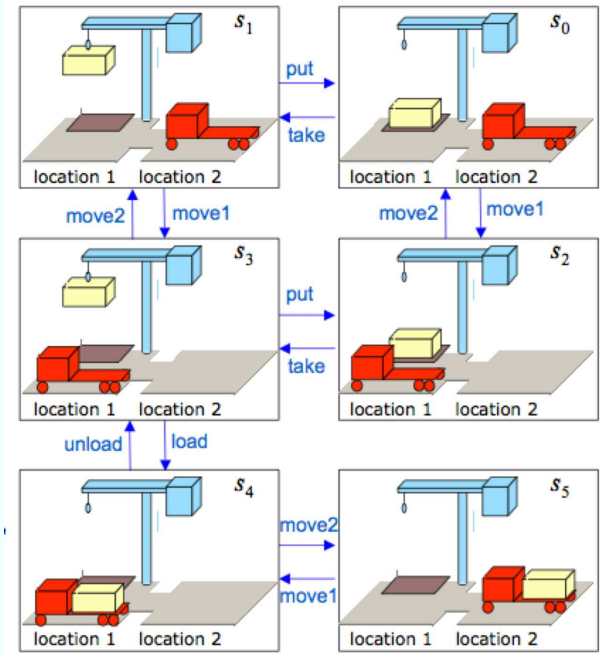


- *But do we need such values of $|V|$?!*

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Why planning is difficult?

- Generalize the earlier example:
  - Five locations, three robot carts, 100 containers, three piles
  - $|V| \approx 10^{277}$
- The number of atoms in the universe is only about 10^{87}
  - The state space in our example is more than 10^{109} times as large (uppss ...)

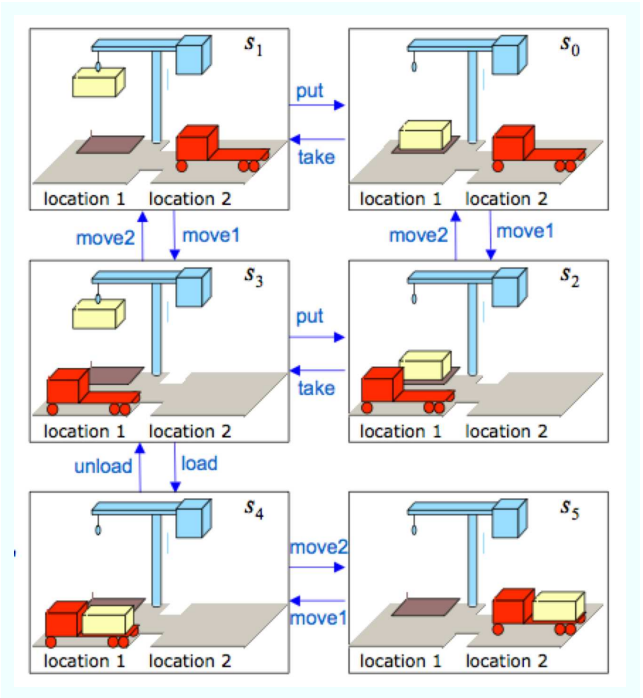


Good news:
Very much not hopeless!

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Why planning is difficult?

- Generalize the earlier example:
  - Five locations, three robot carts, 100 containers, three piles
  - $|V| \approx 10^{277}$
- The number of atoms in the universe is only about 10^{87}
  - The state space in our example is more than 10^{109} times as large (uppss ...)



Good news:
Very much not hopeless!

What class of problems we are in?

Properties

- dynamics: deterministic, nondeterministic or probabilistic
- observability: full, partial, or none
- ...

Deterministic dynamics

Action + current state uniquely determine successor state.

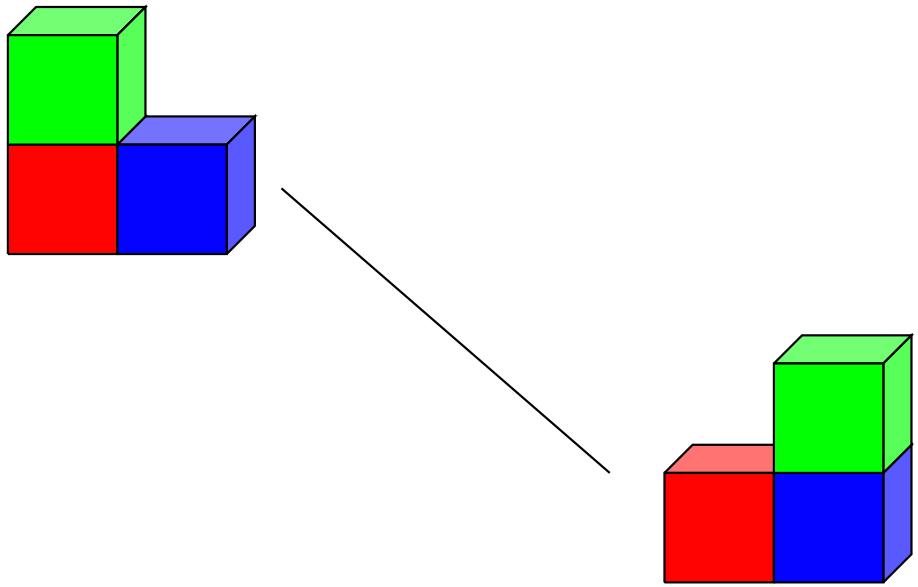
Full observability

Observations/sensing determine current world state uniquely.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Dynamics
Objectives
Back to our problems
State-space search
Uninformed search

Deterministic dynamics example

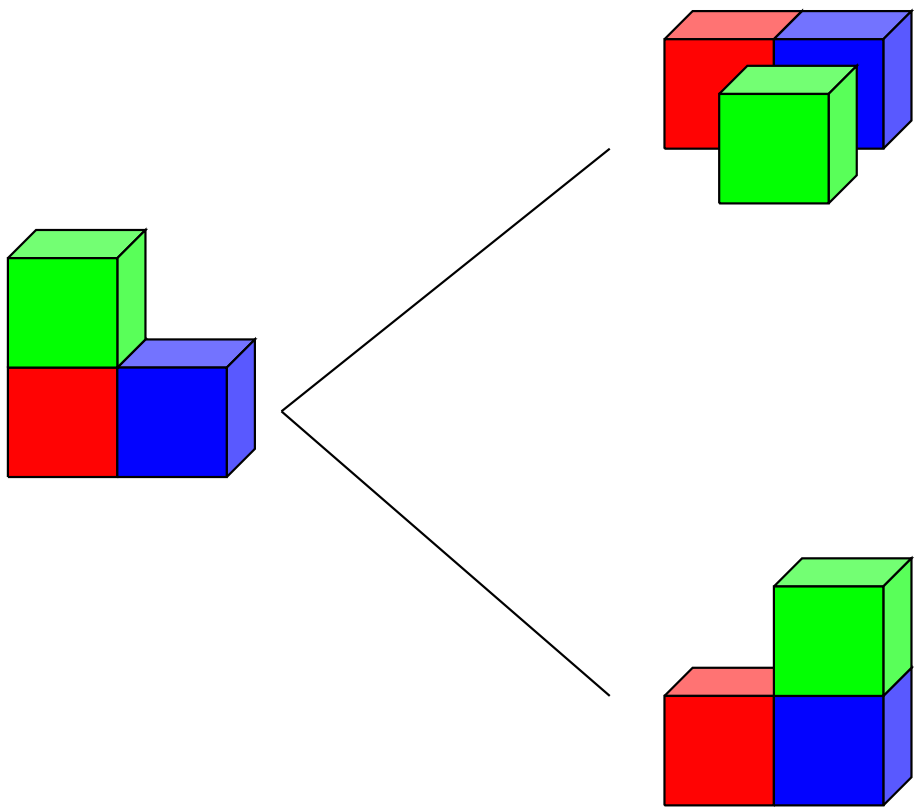
Moving objects with a robotic hand:
move the green block onto the blue block.



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Dynamics
Objectives
Back to our problems
State-space search
Uninformed search

Nondetermistic dynamics example

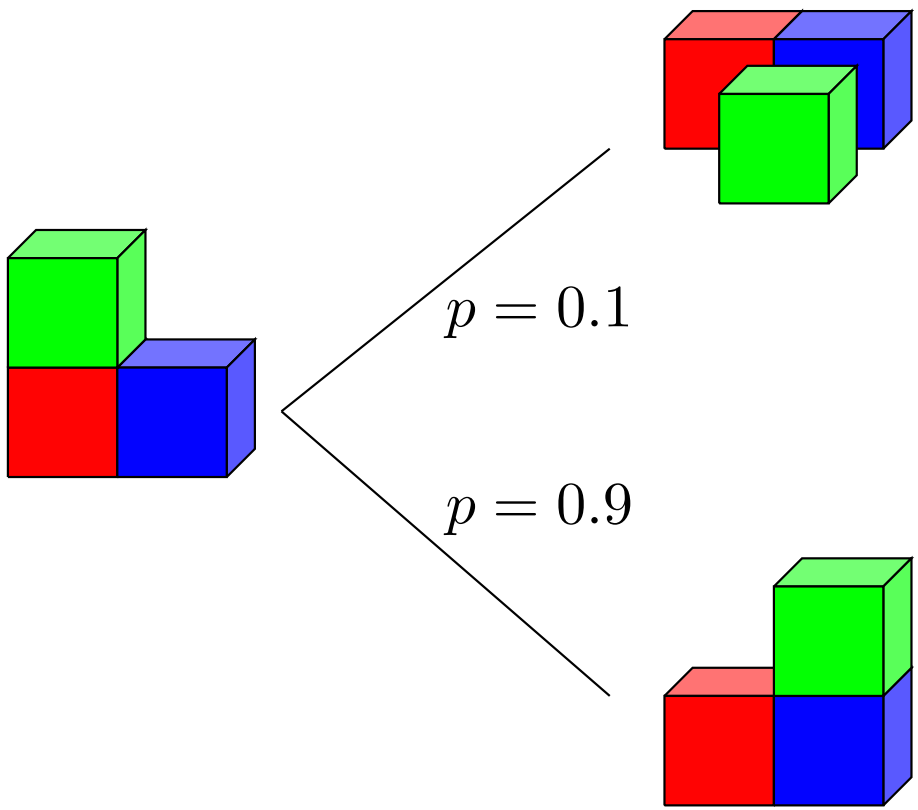
Moving objects with an unreliable robotic hand:
move the green block onto the blue block.



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Dynamics
Objectives
Back to our problems
State-space search
Uninformed search

Probabilistic dynamics example

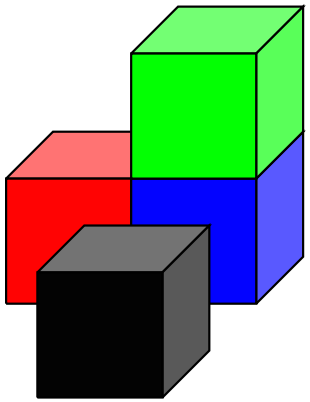
Moving objects with an unreliable robotic hand: move the green block onto the blue block.



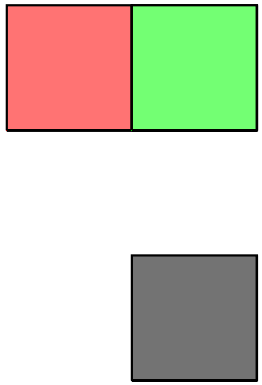
Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Dynamics
Objectives
Back to our problems
State-space search
Uninformed search

What difference does observability make?

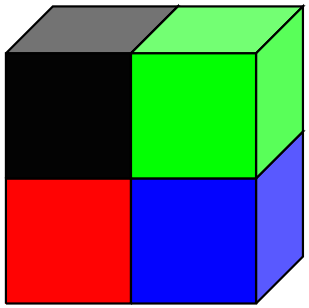
Camera A



Camera B



Goal



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Dynamics
Objectives
Back to our problems
State-space search
Uninformed search

Different objectives

1. Reach a goal state.
   - Example: Earn 500 NIS.
2. Stay in goal states indefinitely (infinite horizon).
   - Example: Never allow the bank account balance to be negative.
3. Maximize the probability of reaching a goal state.
   - Example: To be able to finance buying a house by 2025 study hard and save money.
4. Collect the maximal *expected* rewards/minimal expected costs (infinite horizon).
   - Example: Maximize your future income.
5. ...

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Dynamics
Objectives
Back to our problems
State-space search
Uninformed search

Deterministic transition systems

A transition system is deterministic if there is only one initial state and all actions are deterministic. Hence all future states of the world are completely predictable.

Definition (deterministic transition system)

A deterministic transition system is $\langle S, s_0, A, S_G, c \rangle$ where

- finite state space S
- an initial state $s_0 \in S$
- a set $S_G \subseteq S$ of goal states
- applicable actions $A(s) \subseteq A$ for $s \in S$
- a transition function $s' = f(a, s)$ for $a \in A(s)$
- a cost function $c : A \to [0, \infty)$

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Dynamics
Objectives
Back to our problems
State-space search
Uninformed search

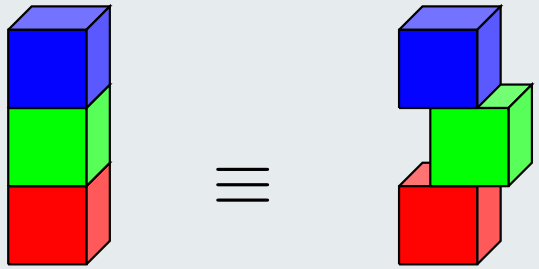
Blocks world

The rules of the game

Location on the table does not matter.



Location on a block does not matter.



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

From problems to models

Deterministic planning

Formalism assumptions

Dynamics

Objectives

Back to our problems

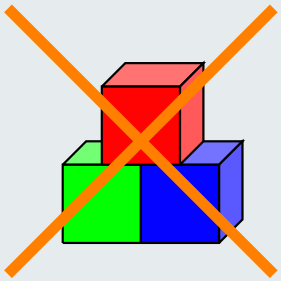
State-space search

Uninformed search

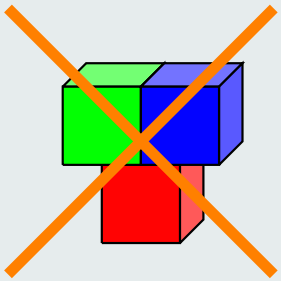
Blocks world

The rules of the game

At most one block may be below a block.



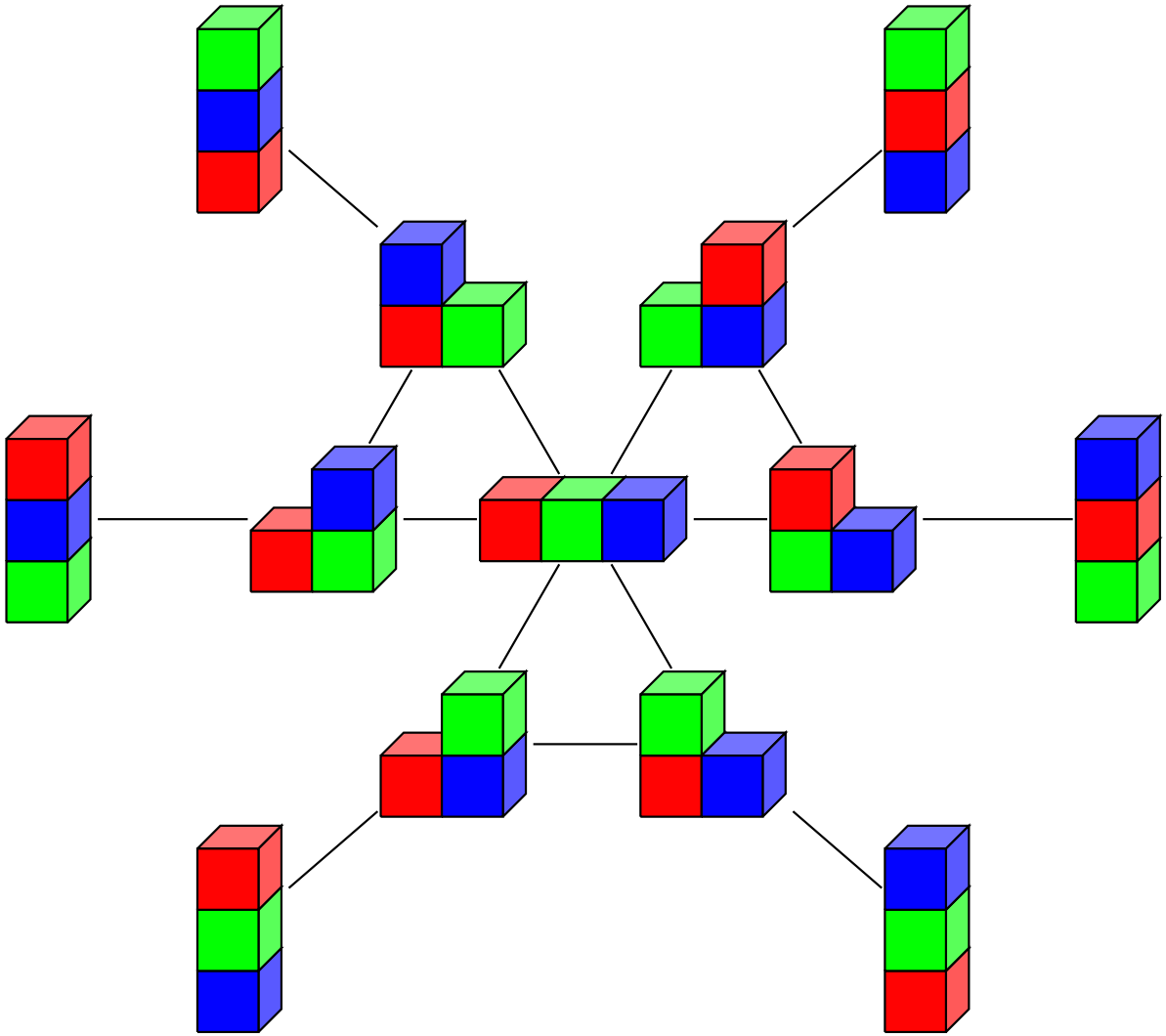
At most one block may be on top of a block.



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Dynamics
Objectives
Back to our problems
State-space search
Uninformed search

Blocks world

The transition graph for three blocks



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

From problems to models

Deterministic planning

Formalism assumptions

Dynamics

Objectives

Back to our problems

State-space search

Uninformed search

Blocks world

Properties

blocks	states
1	1
2	3
3	13
4	73
5	501
6	4051
7	37633
8	394353
9	4596553
10	58941091

blocks	states
11	824073141
12	12470162233
13	202976401213
14	3535017524403
15	65573803186921
16	1290434218669921
17	26846616451246353
18	588633468315403843
19	13564373693588558173
20	327697927886085654441

1. Finding a solution is polynomial time in the number of blocks (move everything onto the table and then construct the goal configuration).
2. Finding a shortest solution is NP-complete (for a compact description of the problem).

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Dynamics
Objectives
Back to our problems
State-space search
Uninformed search

Planning problems: Search for Sequence of Actions

- Route selection (from Arad to Bucharest) (**Waze!**)
- Games (15-puzzle, Chess, Rubik's cube, or ...)
- Selecting and ordering movements of an elevator or a crane
- Production lines control
- Autonomous robots
- Logistics, crisis management (**DART**)
- Cybersecurity (automated penetration testing)
- ...

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

From problems to models

Deterministic planning

Formalism assumptions

Back to our problems

State-space search

Uninformed search

State-space search

- State-space search: one of the big success stories of AI
- Must carefully distinguish two different problems:
  - satisficing planning: any solution is OK (although shorter solutions typically preferred)
  - optimal planning: plans must have shortest possible length
- Both are often solved by search, but:
  - details are very different
  - almost no overlap between good techniques for satisficing planning and good techniques for optimal planning
  - many problems that are trivial for satisficing planners are impossibly hard for optimal planners

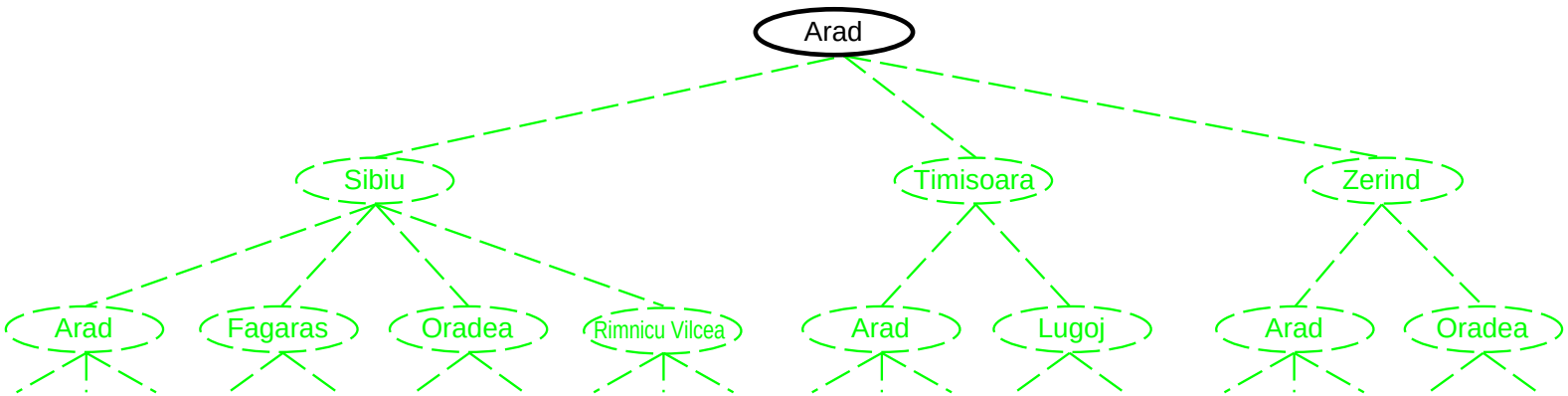
Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Planning by forward search: progression

Progression: Computing the successor state $f(s, a)$ of a state s with respect to an action a.

Progression planners find solutions by forward search:

- start from initial state
- iteratively pick a previously generated state and progress it through an action, generating a new state
- solution found when a goal state generated



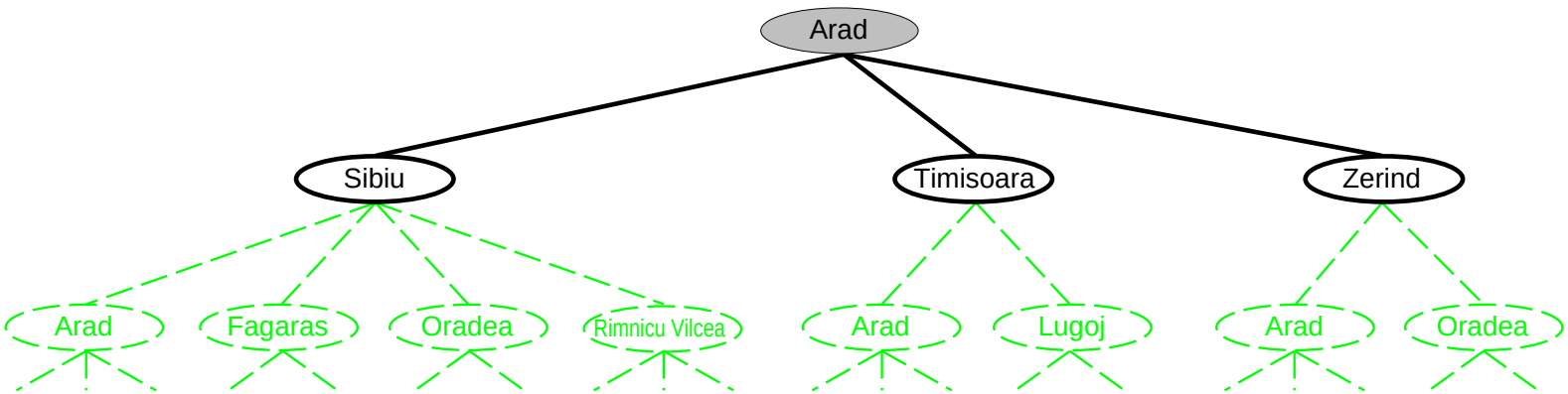
Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Planning by forward search: progression

Progression: Computing the successor state $f(s, a)$ of a state s with respect to an action a.

Progression planners find solutions by forward search:

- start from initial state
- iteratively pick a previously generated state and progress it through an action, generating a new state
- solution found when a goal state generated



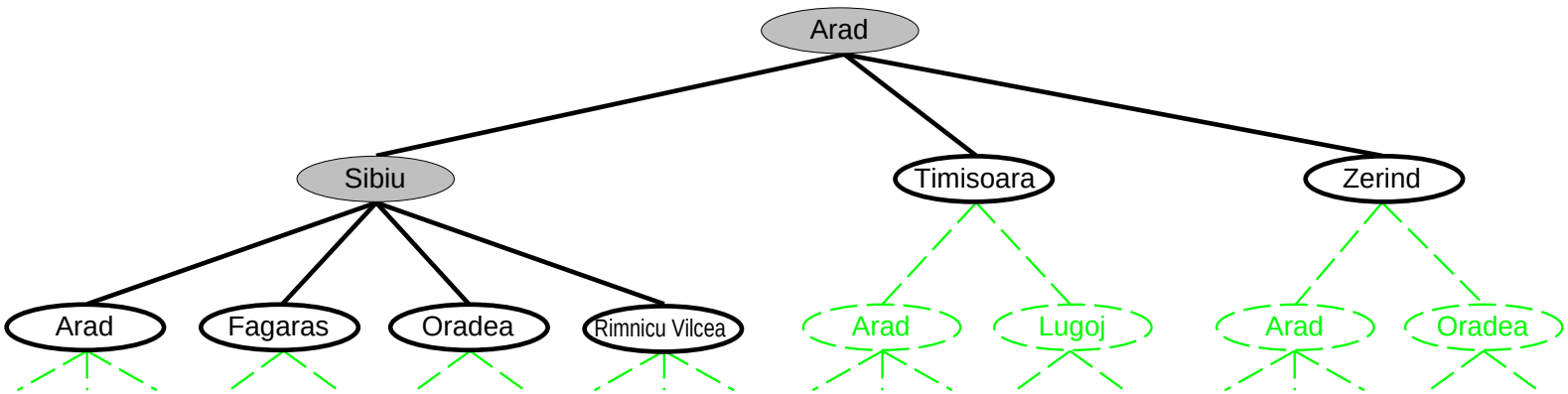
Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Planning by forward search: progression

Progression: Computing the successor state $f(s, a)$ of a state s with respect to an action a.

Progression planners find solutions by forward search:

- start from initial state
- iteratively pick a previously generated state and progress it through an action, generating a new state
- solution found when a goal state generated



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Implementation: search states vs. search nodes

In search, one distinguishes:

- search states $s \rightsquigarrow$ states (vertices) of the transition system
- search nodes $\sigma \rightsquigarrow$ search states plus information on where/when/how they are encountered during search

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Implementation: search states vs. search nodes

In search, one distinguishes:

- search states $s \rightsquigarrow$ states (vertices) of the transition system
- search nodes $\sigma \rightsquigarrow$ search states plus information on where/when/how they are encountered during search

What is in a search node?

Different search algorithms store different information in a search node σ, but typical information includes:

- $\mathit{state}(\sigma)$: associated search state
- $\mathit{parent}(\sigma)$: pointer to search node from which σ is reached
- $\mathit{action}(\sigma)$: an action leading from $\mathit{state}(\mathit{parent}(\sigma))$ to $\mathit{state}(\sigma)$
- $g(\sigma)$: cost of σ (length of path from the root node)

For the root node, $\mathit{parent}(\sigma)$ and $\mathit{action}(\sigma)$ are undefined.

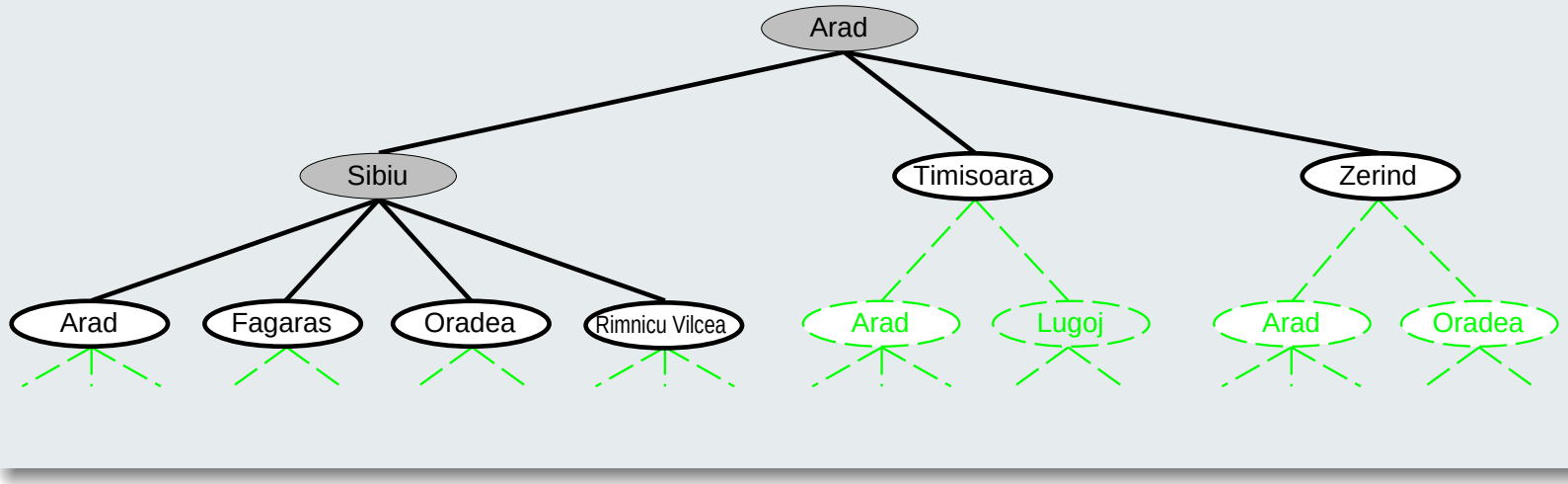
Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Implementation: search states vs. search nodes

In search, one distinguishes:

- search states $s \rightsquigarrow$ states (vertices) of the transition system
- search nodes $\sigma \rightsquigarrow$ search states plus information on where/when/how they are encountered during search

What is in a search node?



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Required ingredients for search

A general search algorithm can be applied to any transition system for which we can define the following three operations:

- init(): generate the initial state
- is-goal(s): test if a given state is a goal state
- succ(s): generate the set of successor states of state s, along with the actions through which they are reached (represented as pairs $\langle o, s' \rangle$ of actions and states)

Together, these three functions form a search space (a very similar notion to a transition system).

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Classification of search algorithms

uninformed search vs. heuristic search:

- uninformed search algorithms only use the basic ingredients for general search algorithms
- heuristic search algorithms additionally use heuristic functions which estimate how close a node is to the goal

systematic search vs. local search:

- systematic algorithms consider a large number of search nodes simultaneously
- local search algorithms work with one (or a few) candidate solutions (search nodes) at a time
- not a black-and-white distinction; there are crossbreeds (e. g., enforced hill-climbing)

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Common procedures for search algorithms

Before we describe the different search algorithms, we introduce three procedures used by all of them:

- make-root-node: Create a search node without parent.
- make-node: Create a search node for a state generated as the successor of another state.
- extract-solution: Extract a solution from a search node representing a goal state.

Procedure make-root-node

make-root-node: Create a search node without parent.

Procedure make-root-node

def make-root-node(s):
    $\sigma :=$ **new** node
    $\mathit{state}(\sigma) := s$
    $\mathit{parent}(\sigma) :=$ undefined
    $\mathit{action}(\sigma) :=$ undefined
    $g(\sigma) := 0$
    return σ

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Procedure make-node

make-node: Create a search node for a state generated as the successor of another state.

Procedure make-node

def make-node(σ, o, s):
$\sigma' :=$ **new** node
$\mathit{state}(\sigma') := s$
$\mathit{parent}(\sigma') := \sigma$
$\mathit{action}(\sigma') := o$
$g(\sigma') := g(\sigma) + 1$ (or $g(\sigma') := g(\sigma) + c(o)$)
return σ'

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search

Procedure extract-solution

extract-solution: Extract a solution from a search node representing a goal state.

Procedure extract-solution

def extract-solution(σ):
    solution := **new** list
    while *parent*(σ) is defined:
        solution.push-front(*action*(σ))
        σ := *parent*(σ)
    return *solution*

Uninformed search algorithms

Popular uninformed systematic search algorithms:

- breadth-first search
- depth-first search
- iterated depth-first search

Popular uninformed local search algorithms:

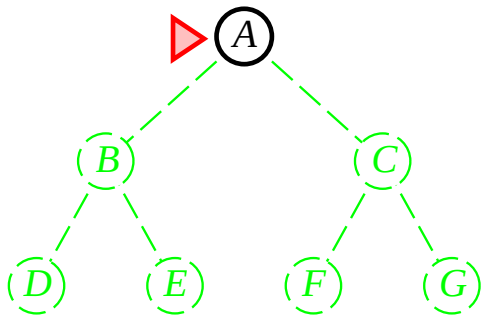
- random walk

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Breadth-first search

Breadth-first search

queue := **new** fifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-back(σ')
return unsolvable

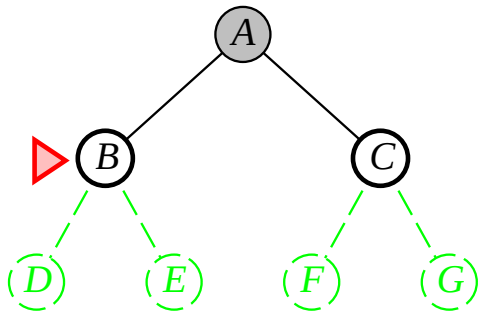


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Breadth-first search

Breadth-first search

queue := **new** fifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-back(σ')
return unsolvable

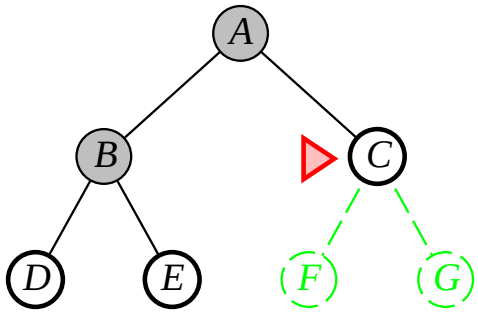


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Breadth-first search

Breadth-first search

queue := **new** fifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-back(σ')
return unsolvable

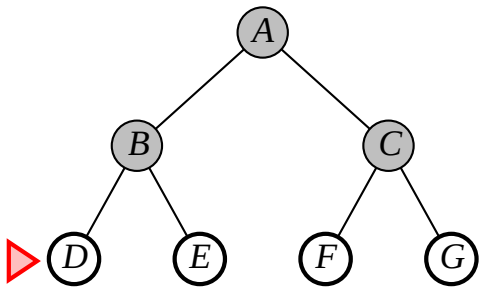


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Breadth-first search

Breadth-first search

queue := **new** fifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-back(σ')
return unsolvable



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Is it any good?

Dimensions for evaluation

- completeness: always find a solution if one exists?
- time complexity: number of nodes generated/expanded
- space complexity: number of nodes in memory
- optimality: does it always find a least-cost solution?

Time/space complexity measured in terms of

b maximum branching factor of the search tree

d depth of the least-cost solution

m maximum depth of the state space (may be ∞)

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Properties of breadth-first search

Complete	Yes (if b is finite)
Time	$1 + b + b^2 + b^3 + \ldots + b^d = O(b^{d+1})$, i.e., $\exp(d)$
Space	$O(b^{d+1})$ *(why?)*
Optimal	Yes (if cost $= 1$ per step); can be generalized *(how?)*

Space is the big problem; can easily generate nodes at 100MB/sec so 24hrs $=$ 8640GB.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Breadth-first search

Breadth-first search

queue := **new** fifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-back(σ')
return unsolvable

- Possible improvement: duplicate detection (see next slide).
- Another possible improvement: test if σ' is a goal node; if so, terminate immediately. (We don't do this because it obscures the similarity to some of the later algorithms.)

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Breadth-first search with duplicate detection

Breadth-first search with duplicate detection

```
queue := new fifo-queue
queue.push-back(make-root-node(init()))
closed := ∅
while not queue.empty():
    σ = queue.pop-front()
    if state(σ) ∉ closed:
        closed := closed ∪ {state(σ)}
        if is-goal(state(σ)):
            return extract-solution(σ)
        for each ⟨o, s⟩ ∈ succ(state(σ)):
            σ′ := make-node(σ, o, s)
            queue.push-back(σ′)
return unsolvable
```

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Duplicate detection: Worth the effort?

Worst-case complexity

Time $1 + b + b^2 + b^3 + \ldots + b^d = O(b^{d+1})$

Same! ☺ *(why?)*

Space $O(b^{d+1})$

Same! ☹ *(why?)*

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Duplicate detection: Worth the effort?

Worst-case complexity

Time $1 + b + b^2 + b^3 + \ldots + b^d = O(b^{d+1})$

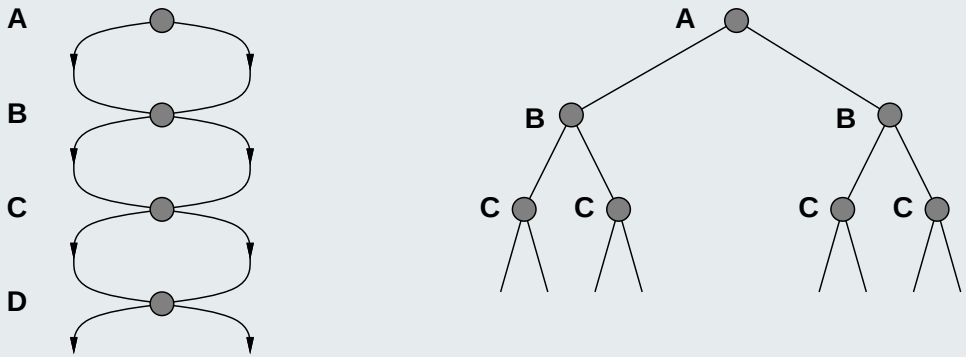
Same! ☺ *(why?)*

Space $O(b^{d+1})$

Same! ☹ *(why?)*

Empirical reality

Failure to detect repeated states can turn a linear problem into an exponential one!

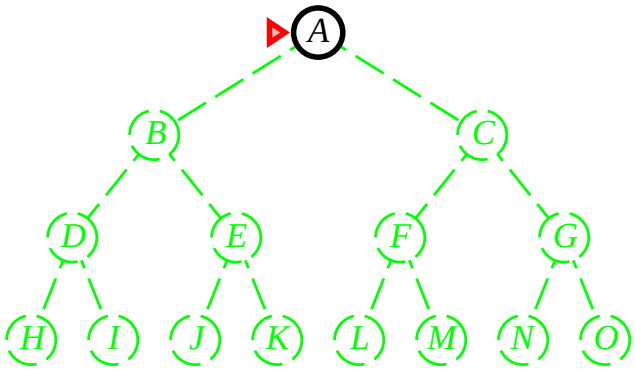


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Depth-first search

Depth-first search

queue := **new** lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-front(σ')
return unsolvable

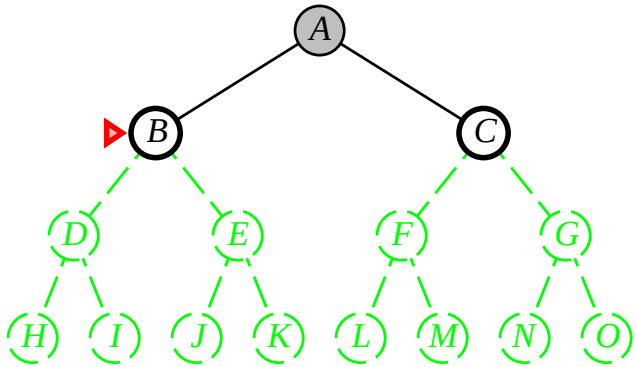


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Depth-first search

Depth-first search

queue := **new** lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-front(σ')
return unsolvable

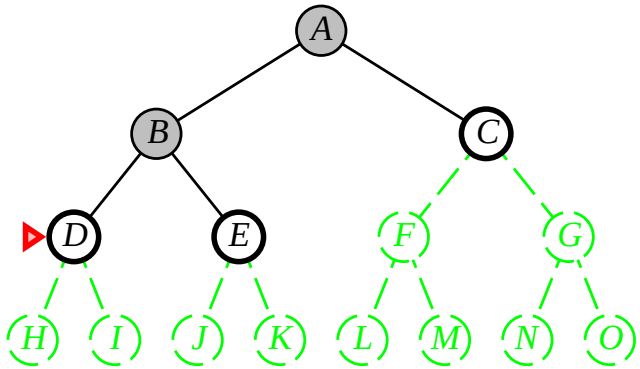


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Depth-first search

Depth-first search

queue := **new** lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-front(σ')
return unsolvable

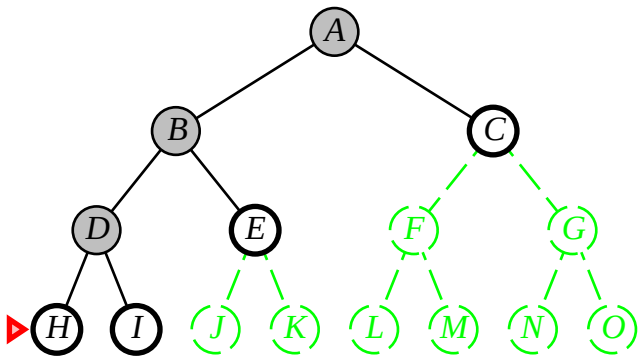


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Depth-first search

Depth-first search

queue := **new** lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-front(σ')
return unsolvable

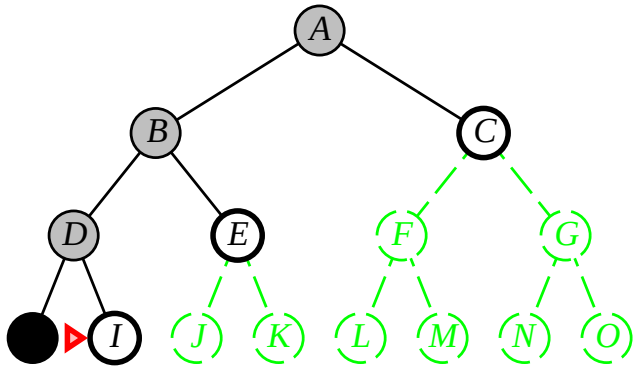


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Depth-first search

Depth-first search

queue := **new** lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-front(σ')
return unsolvable

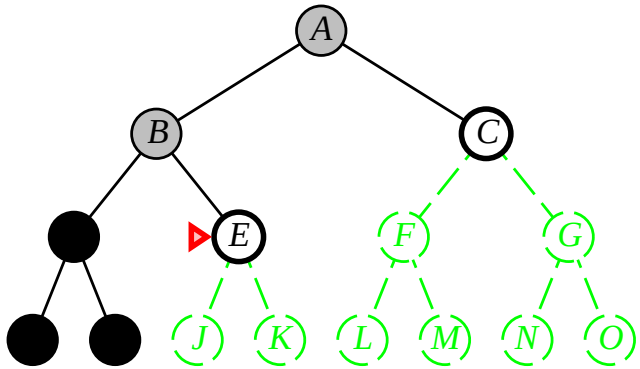


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Depth-first search

Depth-first search

queue := **new** lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-front(σ')
return unsolvable

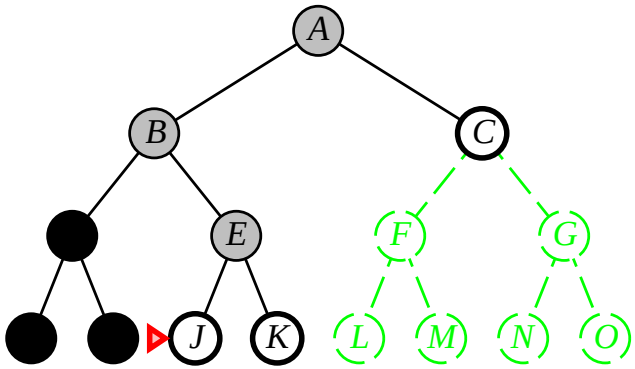


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Depth-first search

Depth-first search

queue := **new** lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-front(σ')
return unsolvable

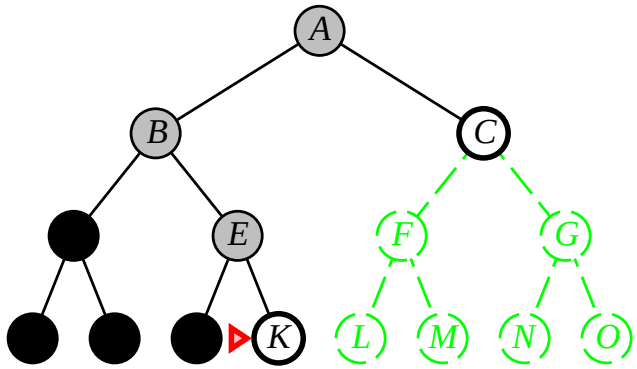


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Depth-first search

Depth-first search

queue := **new** lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-front(σ')
return unsolvable

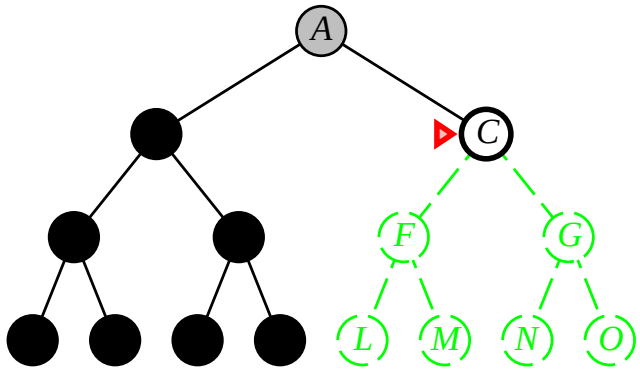


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Depth-first search

Depth-first search

queue := **new** lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-front(σ')
return unsolvable

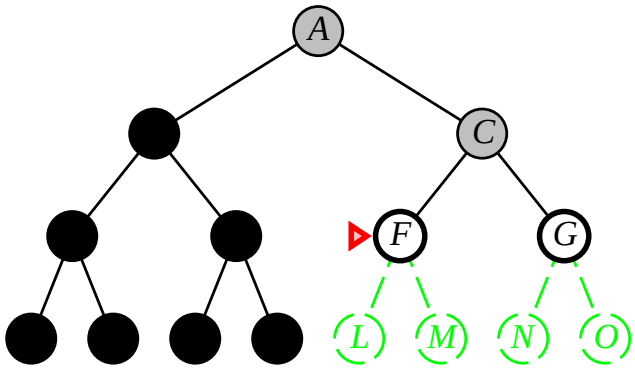


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Depth-first search

Depth-first search

queue := **new** lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-front(σ')
return unsolvable

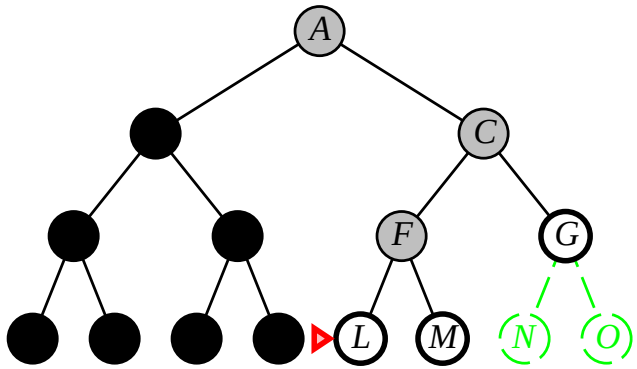


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Depth-first search

Depth-first search

queue := **new** lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-front(σ')
return unsolvable

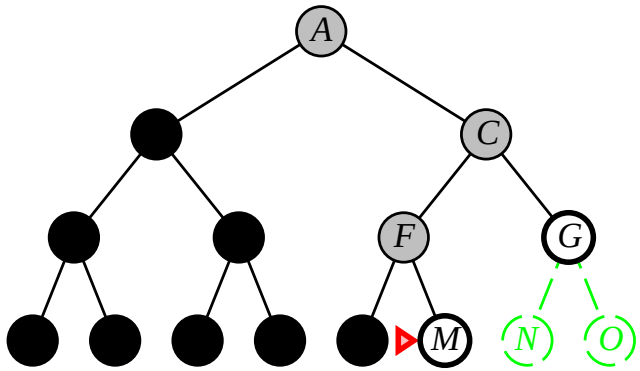


Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Depth-first search

Depth-first search

queue := **new** lifo-queue
queue.push-back(make-root-node(init()))
while not *queue*.empty():
    σ = *queue*.pop-front()
    if is-goal(state(σ)):
        return extract-solution(σ)
    for each $\langle o, s \rangle \in$ succ(*state*(σ)):
        σ' := make-node(σ, o, s)
        queue.push-front(σ')
return unsolvable



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk

Properties of depth-first search

Complete No: fails in spaces with $b = \infty$ or $m = \infty$, spaces with loops

- Possible improvement: duplicate detection along path $\rightsquigarrow$ complete in finite spaces

Time $O(b^m)$

- terrible if m is much larger than d, but
- if solutions are dense, may be much faster than BFS

Space $O(bm) \rightsquigarrow$ linear space!

Optimal No *(right?)*

Random walk

Random walk

$\sigma :=$ make-root-node(init())
forever:
    if is-goal(state(σ)):
        return extract-solution(σ)
    Choose a random element $\langle o, s \rangle$ from succ(state(σ)).
    $\sigma :=$ make-node(σ, o, s)

- The algorithm usually does not find any solutions, unless almost every sequence of actions is a plan.
- Often, it runs indefinitely without making progress.
- It can also fail by reaching a dead end, a state with no successors. This is a weakness of many local search approaches.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
From problems to models
Deterministic planning
Formalism assumptions
Back to our problems
State-space search
Uninformed search
BFS
DFS
Random walk